

CASHFLOW PLANNER — DOCUMENTO MAESTRO PARA CREAR LA APP

1. Objetivo de la app

Crear una app llamada CashFlow Planner para planificación familiar/personal de flujo de caja.

La app debe responder tres preguntas principales:

1. ¿Cuánto dinero tengo disponible?
2. ¿Qué pagos tengo que hacer?
3. ¿Cuánto dinero me quedará después de pagar?

No es una app bancaria.

No es una app de deudas.

No se conecta a bancos.

No usa Plaid.

No usa Firebase.

No usa Supabase.

No requiere internet.

No requiere login.

Debe funcionar 100% offline.

2. Problema que debe resolver

El usuario recibe ingresos variables, con fechas variables y montos variables.

Los gastos mensuales tienen fechas aproximadas y montos que pueden cambiar.

El usuario necesita:

- Registrar ingresos.
- Registrar pagos planificados.
- Modificar ingresos si hubo error o ajuste.
- Modificar pagos si cambió el monto o la fecha.
- Marcar pagos como realizados.
- Ver pagos pendientes.
- Ver pagos ya realizados.
- Ver cuánto queda después de pagar.
- Ver el saldo proyectado.
- Evitar perder la información al cerrar la app.

3. Plataforma recomendada

Opción principal recomendada

React Native + Expo + TypeScript + SQLite local.

Ventajas:

- Sirve para Android y iOS.
- Puede probarse con Expo.
- Puede compilarse luego con EAS Build.
- SQLite guarda datos localmente.
- No depende de internet.
- El código puede estar en GitHub o descargarse como ZIP.

Opción alternativa si se quiere evitar Apple Developer

PWA offline usando React + Vite + IndexedDB.

Ventajas:

- Funciona en iPhone desde Safari.
- Se puede instalar en pantalla de inicio.
- No requiere pagar Apple Developer.
- Puede trabajar offline.
- Se puede alojar gratis.

Desventajas:

- No es app nativa real.
- Algunas funciones como notificaciones, cámara o exportación pueden tener limitaciones en iOS.

Opción alternativa avanzada

Flutter + SQLite local.

Ventajas:

- Muy buena para app nativa.
- Código más portable.
- Buen rendimiento.

Desventajas:

- Curva técnica mayor.
- También requiere Apple Developer para distribución nativa en iPhone.

4. Requisito crítico de almacenamiento

La app no debe guardar información solamente en React state.

Toda la información del usuario debe guardarse en SQLite local.

Cuando la app abre:

- Cargar datos desde SQLite.

Cuando el usuario crea, modifica, elimina o marca algo como pagado:

- Guardar inmediatamente en SQLite.

Los datos deben permanecer aunque:

- Se cierre la app.
- Se reinicie el teléfono.
- Se cierre Expo Go.
- Se vuelva a abrir la app.

5. Arquitectura general

Usar una arquitectura limpia y modular.

Estructura sugerida:

/src /app App.tsx navigation.tsx

/screens HomeScreen.tsx IncomeScreen.tsx PaymentsScreen.tsx PaymentDetailScreen.tsx
BalanceScreen.tsx ProjectedBalanceScreen.tsx CalendarScreen.tsx SummaryScreen.tsx SavingsScreen.tsx
ReceiptScreen.tsx SettingsScreen.tsx

/components Button.tsx Card.tsx MoneyText.tsx TransactionRow.tsx PaymentStatusBadge.tsx
EmptyState.tsx SectionHeader.tsx

/database database.ts migrations.ts seed.ts

/services incomeService.ts paymentService.ts balanceService.ts expenseService.ts savingsService.ts
settingsService.ts notificationService.ts exportService.ts

/logic calculateProjectedBalance.ts groupPaymentsByIncome.ts calculateMonthlySummary.ts
calculateSafeToSave.ts

/types models.ts

/utils dateUtils.ts moneyUtils.ts validation.ts

6. Base de datos SQLite

Usar SQLite local como base de datos principal.

Tabla settings

Campos:

- key TEXT PRIMARY KEY
- value TEXT

Ejemplos:

- currency = USD
 - emergency_cushion = 500
 - notification_days_before = 1
 - notification_time = 09:00
 - language = es
-

Tabla account_balance

Campos:

- id TEXT PRIMARY KEY
- balance REAL
- updated_at TEXT
- notes TEXT

Debe permitir modificar el saldo actual manualmente.

Tabla income

Campos:

- id TEXT PRIMARY KEY
- name TEXT
- amount REAL
- date TEXT
- recurrence TEXT

- status TEXT
- notes TEXT
- created_at TEXT
- updated_at TEXT

Recurrence:

- none
- weekly
- biweekly
- monthly

Status:

- expected
- received
- cancelled

Tabla payments

Campos:

- id TEXT PRIMARY KEY
- name TEXT
- amount REAL
- due_date TEXT
- category_id TEXT
- recurrence TEXT
- status TEXT
- priority TEXT
- notes TEXT
- paid_at TEXT
- created_at TEXT
- updated_at TEXT

Status:

- pending
- paid
- skipped

Priority:

- high
 - medium
 - low
-

Tabla expenses

Campos:

- id TEXT PRIMARY KEY
 - merchant TEXT
 - amount REAL
 - date TEXT
 - category_id TEXT
 - notes TEXT
 - receipt_image_path TEXT
 - created_at TEXT
 - updated_at TEXT
-

Tabla savings_transfers

Campos:

- id TEXT PRIMARY KEY
- amount REAL
- date TEXT
- status TEXT
- notes TEXT
- created_at TEXT
- updated_at TEXT

Status:

- planned
 - completed
 - cancelled
-

Tabla categories

Campos:

- id TEXT PRIMARY KEY
- name TEXT
- color TEXT
- type TEXT

Categorías iniciales:

- Housing
- Vehicles

- Insurance
 - Utilities
 - Credit Cards
 - Food
 - Pets
 - Health
 - Subscriptions
 - Family
 - Savings
 - Other
-

7. Pantallas principales

7.1 Inicio

Debe mostrar:

- Saldo actual.
- Próximo ingreso.
- Próximos pagos.
- Total pagado.
- Total por pagar.
- Saldo proyectado.
- Disponible para ahorro.
- Advertencia si el saldo proyectado será negativo.

Botones:

- Agregar ingreso.
- Agregar pago.
- Realizar pago.
- Escanear recibo.
- Ver resumen mensual.
- Ver calendario.
- Configuración.

El usuario siempre debe poder volver a esta pantalla.

7.2 Ingresos

Debe permitir:

- Agregar ingreso.
- Editar ingreso.
- Eliminar ingreso.

- Cambiar monto.
- Cambiar fecha.
- Marcar como recibido.
- Ver ingresos recibidos.
- Ver ingresos pendientes.
- Ver total mensual.

Campos:

- Nombre.
 - Monto.
 - Fecha.
 - Repetición.
 - Estado.
 - Notas.
-

7.3 Pagos

Debe tener dos secciones visualmente diferentes:

1. Por pagar.
2. Pagado.

La sección “Por pagar” debe verse claramente pendiente, usando rojo/naranja.

La sección “Pagado” debe verse claramente completada, usando verde.

Cada pago debe mostrar:

- Nombre.
- Monto.
- Fecha aproximada.
- Categoría.
- Estado.
- Botón Realizar pago.
- Botón Editar.
- Botón Omitir.

Al final debe mostrar:

- Total pagado.
 - Total por pagar.
 - Saldo después de pagos realizados.
 - Saldo proyectado después de pagos pendientes.
-

7.4 Realizar pago

Cuando el usuario toca "Realizar pago":

1. Cambiar estado de pending a paid.
 2. Guardar en SQLite inmediatamente.
 3. Registrar paid_at.
 4. Mover visualmente de "Por pagar" a "Pagado".
 5. Recalcular saldo proyectado.
 6. Cancelar notificación local de ese pago.
-

7.5 Saldo proyectado

Si el usuario toca "Saldo proyectado", abrir una pantalla de detalle.

Debe mostrar la fórmula:

Saldo actual

- Ingresos
- Pagos realizados
- Pagos pendientes
- Gastos reales
- Transferencias a ahorro = Saldo proyectado

También debe mostrar una lista cronológica de transacciones con saldo después de cada una.

7.6 Calendario

Debe mostrar el mes con:

- Ingresos.
- Pagos.
- Gastos.
- Ahorros.
- Saldo proyectado por día.

Al tocar un día:

- Mostrar transacciones de ese día.
 - Permitir agregar ingreso.
 - Permitir agregar pago.
 - Permitir agregar gasto.
-

7.7 Resumen mensual

Debe mostrar:

- Total de ingresos.
 - Total pagado.
 - Total pendiente.
 - Total de gastos reales.
 - Total transferido a ahorro.
 - Balance neto.
 - Saldo proyectado final.
 - Gastos por categoría.
-

7.8 Ahorro

Debe calcular cuánto es seguro mover a ahorro.

Fórmula:

Disponible para ahorro = saldo proyectado

- pagos pendientes antes del próximo ingreso
- colchón de emergencia

El colchón de emergencia debe ser configurable.

Valor inicial:

\$500.

7.9 Recibos / gastos reales

La app debe permitir registrar gastos reales.

Versión 1:

- Entrada manual.
- Foto opcional del recibo.
- OCR opcional si es fácil, pero no obligatorio.

Campos:

- Comercio.
- Monto.
- Fecha.

- Categoría.
 - Nota.
 - Imagen opcional.
-

7.10 Configuración

Debe permitir:

- Moneda.
- Colchón de emergencia.
- Hora de notificación.
- Días antes para notificar.
- Exportar datos.
- Resetear datos locales.
- Idioma.

Idioma inicial:

Español.

8. Lógica de cálculo

8.1 Reglas

Ingreso suma dinero.

Pago realizado resta dinero.

Pago pendiente también resta del saldo proyectado.

Gasto real resta dinero.

Transferencia a ahorro resta dinero.

Pago omitido no debe restar.

8.2 Fórmula de saldo proyectado

Saldo proyectado = saldo actual

- ingresos del período
- pagos realizados
- pagos pendientes

- gastos reales
 - transferencias a ahorro
-

8.3 Función principal

Crear función:

`calculateProjectedBalance(startingBalance, transactions)`

La función debe:

1. Ordenar transacciones por fecha.
 2. Sumar ingresos.
 3. Restar pagos.
 4. Restar gastos.
 5. Restar ahorros.
 6. Calcular `running_balance`.
 7. Calcular `lowest_balance`.
 8. Calcular `ending_balance`.
 9. Detectar si el saldo será negativo.
-

9. Agrupación por cheque / ingreso

Los ingresos definen períodos.

Ejemplo:

Ingreso Junio 15. Siguiendo ingreso Junio 30.

Los pagos entre Junio 15 y Junio 29 pertenecen al período del cheque de Junio 15.

Debe permitir:

- Mover pago a otra fecha.
 - Cambiar monto.
 - Marcar pagado.
 - Omitir pago.
 - Agregar nota.
-

10. Notificaciones locales

Usar notificaciones locales, no servidor.

Para cada pago pendiente:

- Notificar 1 día antes.
 - Notificar el mismo día.
 - Cancelar notificación si el pago se marca como pagado u omitido.
-

11. Exportación

La app debe exportar:

1. PDF.
2. CSV compatible con Excel.

El PDF debe incluir:

- Mes.
- Saldo inicial.
- Lista de ingresos.
- Lista de pagos.
- Gastos reales.
- Transferencias a ahorro.
- Resumen por categoría.
- Saldo proyectado final.

El CSV debe tener:

- Fecha.
 - Tipo.
 - Nombre.
 - Monto.
 - Categoría.
 - Estado.
 - Notas.
-

12. Reglas de navegación

La app debe usar tabs inferiores:

1. Inicio.
2. Calendario.
3. Pagos.
4. Escanear.
5. Resumen.

Desde cualquier pantalla debe existir forma clara de volver a Inicio.

Todos los botones deben funcionar.

No debe haber botones decorativos sin acción.

13. Diseño visual

Debe ser:

- Limpio.
- Mobile-first.
- Botones grandes.
- Texto claro.
- Español primero.
- Sin elementos confusos.
- Sin anuncios.
- Sin lenguaje bancario complicado.

Colores:

- Verde: ingresos / pagado.
 - Rojo: pagos pendientes.
 - Azul: ahorro.
 - Gris: gastos reales.
 - Amarillo/naranja: advertencias.
-

14. Funcionalidades de versión 1

Incluir:

- SQLite local.
- Agregar ingresos.
- Editar ingresos.
- Eliminar ingresos.
- Agregar pagos.
- Editar pagos.
- Eliminar pagos.
- Marcar pago como realizado.
- Omitir pago.
- Modificar saldo actual.
- Saldo proyectado.
- Pagos pagados vs pendientes.
- Total pagado.
- Total pendiente.
- Calendario o lista mensual.

- Resumen mensual.
- Ahorro sugerido.
- Entrada manual de recibos.
- Exportar PDF.
- Exportar CSV.
- Notificaciones locales.

No incluir en versión 1:

- Login.
 - Bancos.
 - Plaid.
 - Firebase.
 - Supabase.
 - Cloud sync.
 - Cuentas familiares compartidas.
 - Tarjetas de crédito como deuda.
 - Intereses.
 - Inversiones.
 - Complejidad tipo envelope budgeting.
-

15. Metodología de desarrollo

Desarrollar por fases.

Fase 1

- Crear proyecto.
- Configurar navegación.
- Configurar SQLite.
- Crear tablas.
- Crear categorías iniciales.

Fase 2

- Crear módulo de saldo.
- Crear módulo de ingresos.
- Crear módulo de pagos.

Fase 3

- Crear lógica de saldo proyectado.
- Crear lógica de pagos pagados vs pendientes.
- Crear pantalla de detalle de saldo proyectado.

Fase 4

- Crear pantalla Inicio.
- Crear pantalla Pagos.
- Crear pantalla Ingresos.

Fase 5

- Crear resumen mensual.
- Crear calendario o lista mensual.

Fase 6

- Crear notificaciones locales.

Fase 7

- Crear entrada de recibos/gastos.

Fase 8

- Crear exportación PDF/CSV.

Fase 9

- Pulir diseño.
- Validar botones.
- Probar persistencia.
- Probar cálculos.

16. Pruebas obligatorias

Prueba 1: Persistencia de ingresos

1. Agregar ingreso.
2. Cerrar app.
3. Reabrir app.
4. Confirmar que el ingreso sigue guardado.

Prueba 2: Persistencia de pagos

1. Agregar pago.
2. Cerrar app.
3. Reabrir app.
4. Confirmar que el pago sigue guardado.

Prueba 3: Pago realizado

1. Agregar pago pendiente.
2. Tocar Realizar pago.
3. Confirmar que pasa a Pagado.
4. Cerrar app.
5. Reabrir app.
6. Confirmar que sigue como Pagado.

Prueba 4: Modificación

1. Modificar ingreso.
2. Modificar pago.
3. Cerrar app.
4. Reabrir app.
5. Confirmar que los cambios permanecen.

Prueba 5: Saldo proyectado

Ejemplo:

Saldo inicial: \$4,000

Ingreso: \$4,500

Renta: \$2,800

Carro: \$650

Amex: \$200

FPL: \$180

La app debe mostrar el saldo después de cada transacción.

Prueba 6: Saldo negativo

Saldo inicial: \$1,000

Pago pendiente: \$1,500

La app debe mostrar advertencia de saldo negativo.

Prueba 7: Ahorro sugerido

Saldo proyectado: \$3,000

Pagos pendientes antes del próximo ingreso: \$1,800

Colchón de emergencia: \$500

Disponible para ahorro: \$700.

17. Instrucción crítica para la IA desarrolladora

Priorizar primero:

1. Persistencia real en SQLite.
2. Correctitud de cálculos.
3. Edición de ingresos y pagos.
4. Separación clara de pagado vs pendiente.
5. Navegación funcional.
6. Diseño visual.

No avanzar a funciones decorativas hasta que la app guarde correctamente los datos y calcule correctamente el saldo proyectado.

18. Entregables obligatorios

La IA o desarrollador debe entregar:

1. Código fuente completo.
 2. Archivo ZIP descargable.
 3. Instrucciones para instalar.
 4. Instrucciones para correr localmente.
 5. Instrucciones para probar en Android.
 6. Instrucciones para probar en iPhone.
 7. Base de datos SQLite funcional.
 8. Pruebas básicas documentadas.
 9. Explicación de cómo exportar PDF/CSV.
 10. Confirmación de que los datos persisten después de cerrar la app.
-

19. Opciones recomendadas fuera de Replit

Opción A — React Native + Expo local

Ideal si se quiere mantener la posibilidad de Android/iOS.

Requisitos:

- Computadora.
- Node.js.
- Expo.
- GitHub.
- Código descargable.

Ventaja:

Control total del código.

Desventaja:

Para app iPhone nativa instalable se necesitará Apple Developer eventualmente.

Opción B — PWA offline

Ideal si se quiere evitar pagar Apple inicialmente.

Tecnología:

- React.
- Vite.
- IndexedDB.
- Service Worker.
- PWA installable.

Ventaja:

Puede instalarse en pantalla inicial de iPhone sin App Store.

Desventaja:

No es app nativa real.

Opción C — Flutter local

Ideal si se quiere app nativa seria a futuro.

Tecnología:

- Flutter.
- Dart.
- SQLite.

Ventaja:

Muy sólido para Android/iOS.

Desventaja:

Más técnico y también requiere Apple Developer para distribución iPhone.

20. Decisión recomendada

Para este proyecto, la mejor ruta es:

Primero crear una PWA offline si el objetivo es usarla rápido sin pagar Apple.

Luego, si la app funciona bien y vale la pena, convertirla a React Native/Expo o Flutter para app nativa.

Si se quiere ir directo a app móvil sería:

React Native + Expo + SQLite local, pero exigir desde el inicio código fuente descargable y controlado en GitHub.